

Frontend Interview Handbook — Part 7: Testing, Git, CI/CD & Deployment

SECTION A: TESTING

Q1: What's the testing pyramid, and where does frontend testing typically fall?

Answer: The testing pyramid (bottom to top): **unit tests** (many, fast, test individual functions/components in isolation), **integration tests** (fewer, test how multiple units work together — e.g., a form submitting and updating state), **E2E tests** (fewest, slowest, test full user flows through a real or near-real browser). The shape matters — a healthy suite has many fast unit tests and relatively few slow E2E tests, not the inverse ("ice cream cone" anti-pattern), because E2E tests are expensive to run and maintain, and slow feedback loops hurt velocity.

Q2: How do you test a React component with React Testing Library (RTL)? What's RTL's core philosophy?

Answer: RTL's philosophy: **test behavior the way a user experiences it, not implementation details**. Query elements by role/text/label the way a user would find them (`getByRole`, `getByLabelText`) rather than by internal state, class names, or component instance methods — this makes tests resilient to refactors that don't change user-facing behavior.

```
jsx

import { render, screen, fireEvent } from '@testing-library/react';

test('shows error when submitting empty form', () => {
  render(<LoginForm />);
  fireEvent.click(screen.getByRole('button', { name: /log in/i }));
  expect(screen.getByText(/email is required/i)).toBeInTheDocument();
});
```

Common mistake: Testing internal state directly (`wrapper.state('isOpen')` style, from older Enzyme-era habits) instead of asserting on what's rendered to the user.

Q3: What's the difference between `fireEvent` and `userEvent` in RTL?

Answer: `fireEvent` dispatches a single low-level DOM event directly. `userEvent` simulates a more realistic full sequence of events a real user interaction would trigger (e.g., typing fires `keyDown`, `keyPress`, `input`, `keyUp` for each character, not just one synthetic change

event) — generally preferred as the more accurate/realistic default, especially for anything involving focus, hover, or multi-step interaction.

Q4: How would you test an async component that fetches data on mount?

```
jsx

test('displays user name after fetch resolves', async () => {
  render(<Profile userId="1" />);
  expect(screen.getByText(/loading/i)).toBeInTheDocument();

  const name = await screen.findByText('Samuel Obeng'); // findBy = waits + ret
  expect(name).toBeInTheDocument();
});
```

Explain out loud: `findBy*` queries return a promise and retry until the element appears or a timeout is hit — the correct tool for async UI, vs `getBy*` (synchronous, throws immediately if not found) and `queryBy*` (synchronous, returns `null` if not found — useful for asserting something is *absent*).

Q5: What's the difference between mocking and stubbing, and how would you mock an API call in a test?

Answer: Broadly, both replace real dependencies with controlled fakes for testing in isolation — "mock" is often used loosely to cover both. For API calls, you'd typically mock at the network boundary (using something like MSW — Mock Service Worker — which intercepts actual `fetch`/`axios` calls at the network level, so your component code doesn't need any test-specific branching) or mock the fetching function/module directly with `jest.mock`.

```
js

// MSW handler example
rest.get('/api/user/:id', (req, res, ctx) => {
  return res(ctx.json({ id: req.params.id, name: 'Samuel Obeng' }));
});
```

Why MSW is often preferred: the component code under test doesn't know it's being tested — it makes a real `fetch` call that gets intercepted at the network layer, which is more realistic than deeply mocking internal modules.

Q6: What's the difference between unit testing a pure function vs snapshot testing a component — and what are snapshot tests' pitfalls?

Answer: A pure function unit test asserts a specific, meaningful input/output relationship. A snapshot test captures the rendered output and flags any future difference from a saved baseline. Pitfall: snapshots are easy to accept blindly ("update snapshot" without actually reviewing the diff), turning them into a rubber stamp rather than a real safety net — and they're brittle to unrelated formatting/markup changes, generating noisy false-positive failures. Best practice: use snapshots sparingly, prefer explicit behavioral assertions for anything that actually matters.

SECTION B: GIT

Q7: Explain the difference between `git merge` and `git rebase`. When would you use each?

Answer: `merge` combines two branch histories with a new merge commit, preserving the full, real history of both branches (including the exact order things happened) — safe for shared/public branches. `rebase` replays your branch's commits on top of another branch's latest commit, producing a linear history without a merge commit — cleaner history, but rewrites commit hashes, which is dangerous on a branch others are also working from (their history diverges from yours). Common convention: rebase local feature branches before opening a PR to keep history clean; never rebase a shared/public branch that others have already pulled.

Q8: What's the difference between `git reset`, `git revert`, and `git checkout` (for undoing changes)?

Answer: `git revert` creates a *new* commit that undoes a previous commit's changes — safe for shared history since nothing is rewritten. `git reset` moves the branch pointer (and optionally working directory/staged changes) backward, rewriting history — dangerous if already pushed/shared. `git checkout <commit> -- <file>` restores a specific file to a previous commit's state without touching the rest of the branch's history.

Q9: How do you resolve a merge conflict?

Answer: Git marks the conflicting sections in the file with `<<<<<<`, `=====`, `>>>>>>` markers showing both versions. You manually edit the file to the correct final state, remove the conflict markers, `git add` the resolved file(s), then complete the merge/rebase (`git commit` for a merge, `git rebase --continue` for a rebase). Best practice: understand *why* the conflict happened (usually both branches touched the same lines) rather than blindly picking one side — sometimes the correct resolution combines both changes.

Q10: What's a good commit message/PR practice, and why does it matter on a team?

Answer: Commits should be atomic (one logical change per commit) with a clear, imperative-mood message ("Add user authentication middleware," not "fixed stuff"). Good PR practice: small, reviewable PRs over giant ones, a clear description of *why* (not just what) the change was made, and linking to the relevant ticket/issue. This matters because commit history is documentation — future debugging (`git blame`), bisecting a regression) depends entirely on past commits being meaningful.

SECTION C: CI/CD & DEPLOYMENT

Q11: What is CI/CD, and what does a typical frontend pipeline look like?

Answer: **CI (Continuous Integration):** automatically building and testing every code change (usually on every push/PR) to catch problems early, before merge. **CD (Continuous Deployment/Delivery):** automatically deploying code that passes CI — Delivery means it's ready to deploy with a manual trigger; Deployment means it ships automatically with no human step. Typical frontend pipeline (matches your GitHub Actions experience): lint → type-check → unit tests → build → (optionally) E2E tests against a preview deployment → deploy to staging/production.

```
yaml

# simplified GitHub Actions example
on: [pull_request]
jobs:
  ci:
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npm run test
      - run: npm run build
```

Q12: What's a preview deployment, and why is it valuable for frontend work specifically?

Answer: A preview deployment automatically builds and deploys every PR/branch to its own temporary, shareable URL (common with Vercel/Netlify) — lets reviewers, designers, and stakeholders see and click through the actual change in a real environment before merging, rather than reviewing code alone. Especially valuable for frontend since visual/UX changes are hard to fully evaluate from a diff.

Q13: How would you handle environment variables/secrets across environments (dev/staging/production) in a frontend deployment?

Answer: Never commit secrets to the repo. Use environment-specific `.env` files (gitignored) locally, and the hosting platform's secrets management (Vercel/Netlify env var settings, GitHub Actions secrets) for CI/deployed environments. In Next.js specifically, distinguish between server-only env vars (safe for secrets, never shipped to client) and `NEXT_PUBLIC_`-prefixed vars (bundled into client JS, visible to anyone — genuinely public config only).

Q14: What's a rollback strategy, and why does it matter for production deployments?

Answer: A rollback strategy is the plan for quickly reverting to a previous known-good deployment if a release causes issues in production — ideally near-instant (e.g., re-pointing traffic to the previous build/version on a platform like Vercel) rather than requiring a full revert-commit-and-redeploy cycle, which is much slower during an active incident. Mentioning you'd want fast rollback capability, not just "we'd fix forward," shows production maturity.

Rapid-Fire Round

- **Q: What's the difference between `npm ci` and `npm install`?** A: `npm ci` installs exactly what's in `package-lock.json` (fails if it's out of sync with `package.json`), deletes `node_modules` first, and is faster/more deterministic — preferred for CI environments. `npm install` can update the lockfile and is meant for local dev.
 - **Q: What's a feature flag, and why use one?** A: A runtime toggle that lets you ship code to production without exposing it to users yet, or roll out gradually/to specific segments — decouples deployment from release.
 - **Q: What's blue-green deployment?** A: Running two identical production environments; traffic switches entirely from the old ("blue") to the new ("green") version once verified, enabling instant rollback by switching back.
 - **Q: Why run `npm audit`/dependency scanning in CI?** A: Catches known security vulnerabilities in dependencies before they reach production.
 - **Q: What's the difference between a monorepo and separate repos, and a tradeoff of each?** A: Monorepo — shared tooling/versioning, easier cross-package refactors, but slower CI/larger checkout if not tooled well (Nx/Turborepo). Separate repos — clean isolation, but harder to coordinate changes spanning multiple packages.
-

End of Part 7. Say "next part" for Part 8: Frontend System Design.